

Universidad Tecnológica Nacional

Base de Datos II

Funciones de Usuario

Autor: Angel Simón

Índice

1. Introducción	2
2. Ventajas y Desventajas	2
2.1. Ventajas	2
2.2. Desventajas	3
3. Estructura de una Función de Usuario	3
4. Ejemplo Práctico: Cálculo de Edad	4
4.1. Llamado directo a la función	5
4.2. Uso con datos de una tabla	5

1. Introducción

Las funciones definidas por el usuario (*User-Defined Functions*, UDF) son bloques de código SQL reutilizables que encapsulan lógica compleja con el propósito de realizar cálculos, transformar datos o devolver resultados específicos. Dependiendo del tipo de valor que retornan, pueden clasificarse en dos grandes categorías: funciones escalares, que devuelven un único valor, y funciones de tabla, que devuelven un conjunto de resultados.

En el marco de esta materia, el trabajo se centra en funciones de usuario que retornan valores escalares.

Definición

Una función de usuario es un objeto de base de datos que encapsula lógica reutilizable. Las funciones escalares devuelven un único valor de un tipo de dato determinado, mientras que las funciones de tabla retornan un conjunto de filas.

2. Ventajas y Desventajas

El uso de funciones de usuario conlleva una serie de beneficios y consideraciones que es importante tener en cuenta al momento de diseñar una solución.

2.1. Ventajas

- **Simplificación de consultas:** Permiten extraer la complejidad de un cálculo fuera de la consulta principal, reduciendo su extensión y haciéndola más clara.
- **Legibilidad:** Al delegar parte de la lógica a funciones específicas, el código SQL resulta más fácil de leer y comprender.
- **Reutilización:** Una función bien definida puede invocarse desde múltiples consultas o contextos distintos, evitando la duplicación de lógica.

- **Mantenimiento:** Si varias consultas dependen de una misma función, cualquier modificación en ella se propaga automáticamente a todos los usos, centralizando el mantenimiento.

2.2. Desventajas

- **Rendimiento:** Las funciones de usuario pueden degradar el rendimiento de las consultas cuando operan sobre grandes volúmenes de datos, ya que se ejecutan una vez por cada fila del resultado.
- **Restricciones de escritura:** No es posible ejecutar instrucciones de modificación de datos (INSERT, UPDATE, DELETE) dentro de una función de usuario. Para ese tipo de operaciones se deben utilizar otros mecanismos, como los procedimientos almacenados.

Rendimiento

Dado que las funciones de usuario se ejecutan por cada fila procesada en la consulta, su impacto en el rendimiento puede volverse significativo en tablas con muchos registros. Es conveniente evaluar alternativas cuando se trabaja con grandes volúmenes de datos.

3. Estructura de una Función de Usuario

La sintaxis general para crear una función escalar en SQL Server es la siguiente:

```
CREATE FUNCTION esquema.NombreFuncion(  
    @parametro1 TipoDato,  
    @parametro2 TipoDato  
)  
RETURNS TipoRetorno  
AS  
BEGIN  
    DECLARE @resultado TipoRetorno;  
  
    -- Lógica de la función  
    SET @resultado = ...;  
  
    RETURN @resultado;  
END;
```

Código 1. Estructura general de una función escalar

Los elementos principales que componen una función son:

- **CREATE FUNCTION:** Instrucción que inicia la definición de la función, junto con el esquema y el nombre que la identifica.
- **Parámetros de entrada:** Variables prefijadas con @ que reciben los valores al momento del llamado.
- **RETURNS:** Indica el tipo de dato que la función devolverá.
- **BEGIN / END:** Delimitan el bloque de código que contiene la lógica de la función.
- **RETURN:** Instrucción que finaliza la ejecución y devuelve el valor calculado.

4. Ejemplo Práctico: Cálculo de Edad

A continuación se presenta un ejemplo concreto de una función escalar. La función recibe dos fechas: la fecha de nacimiento de una persona y una fecha de referencia, y retorna la edad de esa persona expresada en años enteros.

```
CREATE FUNCTION dbo.CalcularEdad(  
    @fechaNacimiento DATE,  
    @fechaActual      DATE  
)  
RETURNS INT  
AS  
BEGIN  
    DECLARE @edad INT;  
  
    -- Calcular la diferencia en años entre las dos fechas  
    SET @edad = DATEDIFF(YEAR, @fechaNacimiento, @fechaActual);  
  
    -- Restar un año si la persona aún no cumplió años en el año actual  
    IF (MONTH(@fechaActual) < MONTH(@fechaNacimiento))  
        OR (MONTH(@fechaActual) = MONTH(@fechaNacimiento)  
            AND DAY(@fechaActual) < DAY(@fechaNacimiento))  
    BEGIN  
        SET @edad = @edad - 1;  
    END;  
  
    RETURN @edad;  
END;
```

Código 2. Función para calcular la edad de una persona

La lógica de la función se basa en calcular la diferencia de años con DATEDIFF y luego corregir el resultado en caso de que la persona aún no haya cumplido años en el año de referencia.

4.1. Llamado directo a la función

Una vez creada, la función puede invocarse directamente dentro de una consulta SELECT:

```
SET DATEFORMAT 'YMD';

SELECT dbo.CalcularEdad('2000/10/10', '2024/10/15') AS Edad;
-- Resultado: 24

SELECT dbo.CalcularEdad('2000/10/10', '2024/10/09') AS Edad;
-- Resultado: 23
```

Código 3. Llamado directo a la función con valores literales

En el primer caso, la fecha de referencia es posterior al cumpleaños en el año actual, por lo que la edad calculada es 24. En el segundo caso, la fecha de referencia es anterior al cumpleaños, de modo que la función devuelve 23.

4.2. Uso con datos de una tabla

La función también puede utilizarse sobre los datos almacenados en una tabla. Para ilustrarlo, se define primero la tabla Personas y se insertan algunos registros:

```
CREATE TABLE Personas (
    IDPersona      INT          NOT NULL PRIMARY KEY,
    Nombre         VARCHAR(20)  NOT NULL,
    Apellido       VARCHAR(20)  NOT NULL,
    FechaNacimiento DATE       NOT NULL
);
GO

INSERT INTO Personas (IDPersona, Nombre, Apellido, FechaNacimiento)
VALUES
    (1, 'Juan', 'Gómez', '1990-05-12'),
    (2, 'Ana', 'Martínez', '1985-08-23'),
```

```
(3, 'Luis', 'Fernández', '1992-11-30'),  
(4, 'María', 'Pérez', '1988-03-15'),  
(5, 'Carlos', 'López', '1995-07-09');
```

Código 4. Creación e inserción de datos en la tabla Personas

Luego, se puede consultar la edad actual de cada persona utilizando GETDATE() como fecha de referencia:

```
SELECT  
    Apellido,  
    Nombre,  
    FechaNacimiento,  
    dbo.CalcularEdad(FechaNacimiento, GETDATE()) AS Edad  
FROM Personas;
```

Código 5. Consulta de edad calculada para cada persona

En esta consulta, la función recibe el valor del campo FechaNacimiento como primer argumento y el resultado de GETDATE() como segundo. Dado que la función se evalúa una vez por cada fila retornada, si la tabla contiene muchos registros, el llamado se repetirá la misma cantidad de veces.

Modificación de datos dentro de funciones

Las funciones de usuario no pueden contener instrucciones INSERT, UPDATE ni DELETE. Intentar ejecutarlas dentro de una función resultará en un error de compilación. Para realizar operaciones de modificación de datos, se deben utilizar procedimientos almacenados u otros mecanismos provistos por el motor.